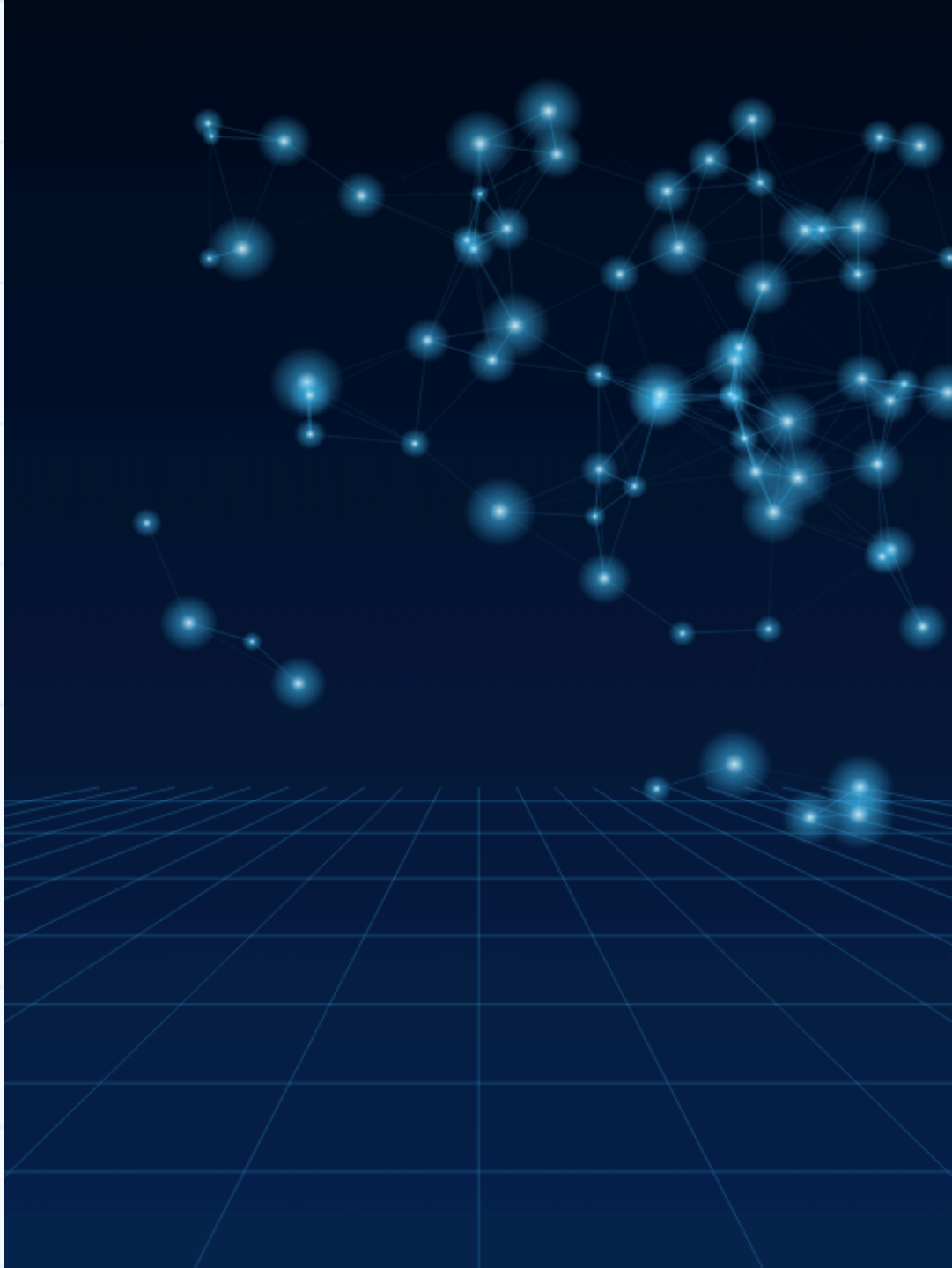
TRABALHO AV2

Inteligência Artificial Computacional

Aprendizado de Máquina: Classificação e Regressão

Alunos: Rodrigo Herrmann e Gustavo Furtado
Professora: Dra. Cynthia Moreira Maia — UNIFOR
Python + NumPy



1. Introdução

Objetivo: comparar algoritmos supervisionados implementados do zero em um problema de classificação e um de regressão do OpenML.

O QUE COMPARAMOS

Classificação: kNN (Euclidiana e Manhattan) e Bayes Univariado.

Regressão: kNN (Euclidiana e Manhattan) e Regressão Linear Múltipla.

POR QUE IMPORTA

Escolher um modelo exige equilibrar qualidade preditiva e custo computacional (tempo de treino e de teste). Implementar do zero, expõe a matemática de cada método.

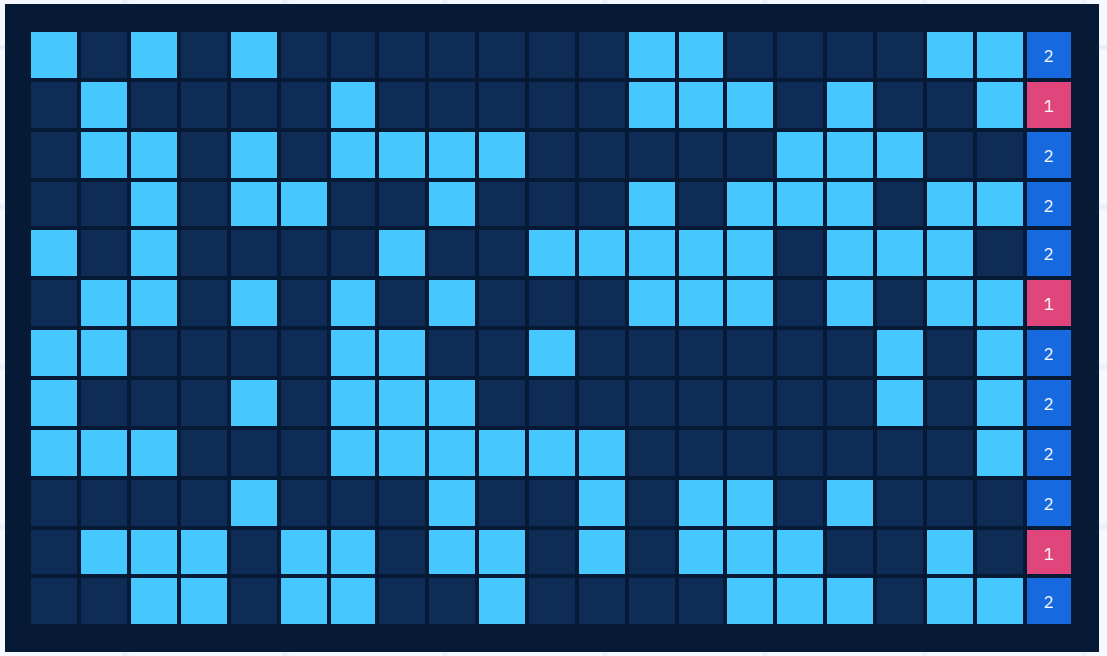
3.1 Banco de Dados — Classificação

AutoUniv-au1-1000

OpenML ID 1547

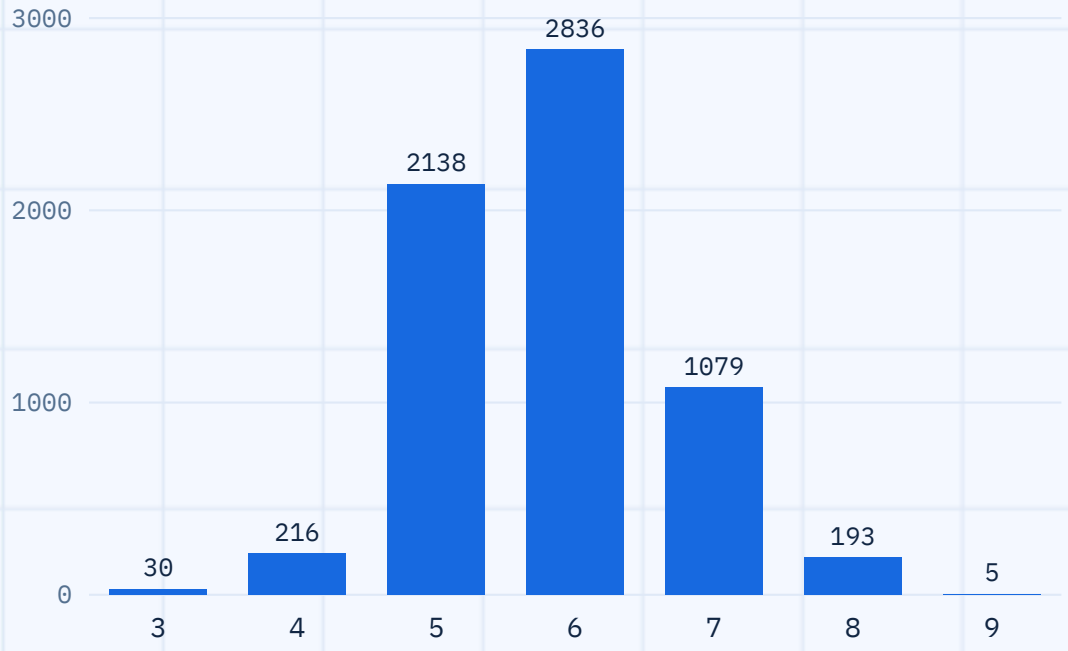
Dataset artificial do gerador AutoUniv (Ray J. Hickey): regras aleatórias com ruído controlado geram exemplos rotulados. Atributos binários (0/1), sem valores ausentes. Classes desbalanceadas: class1 = 25,9% e class2 = 74,1%.

1.000 AMOSTRAS · 20 ATRIBUTOS · 2 CLASSES



As 12 primeiras linhas reais do arquivo: cada quadrado é um atributo (azul-claro = 1), a última coluna é a classe.

3.1 Banco de Dados — Regressão



Quantidade de vinhos por nota de qualidade (6.497 amostras).

Wine Quality

OpenML ID 287

Prever a nota de qualidade (3 a 9) de vinhos a partir de 11 medidas físico-químicas: acidez, açúcar, cloretos, SO_2, densidade, pH, sulfatos, álcool etc.

6.497 AMOSTRAS · 11 ATRIBUTOS · alvo quality (média 5,82 ± 0,87)

Cortez et al., 2009

2. Algoritmos — kNN

k vizinhos.
Uma decisão.

fit: só guarda X e y na memória. Modelo “preguiçoso”, treino ≈ 0 s.

predict: para cada amostra, mede a distância até todo o treino, ordena e pega os k mais próximos.

Classificação: voto majoritário (k = 3).
Regressão: média dos vizinhos (k = 5).

DISTÂNCIAS

Euclidiana $\sqrt{\Sigma(x_i - y_i)^2}$

Manhattan $\Sigma|x_i - y_i|$

src/knn.py

```python
class KNN:
    def fit(self, X_train, y_train):  # treinamento
        self.X_train = X_train
        self.y_train = y_train

    def calculate_predicts(self, x):
        distances = [self.calculate_distances(x, x_train)
                     for x_train in self.X_train]
        k_indices = np.argsort(distances)[:self.k]
        k_nearest_label = [self.y_train[i] for i in k_indices]
        if self.task == "classification":  # voto
            unique, counts = np.unique(k_nearest_label,
                                       return_counts=True)
            return unique[np.argmax(counts)]
        if self.task == "regression":      # média
            return np.mean(k_nearest_label)

    def predict(self, X_test):  # predição
        return [self.calculate_predicts(x) for x in X_test]
```

2. Algoritmos — Classificador Bayesiano Univariado

Uma normal por atributo, por classe.

fit: para cada classe, calcula a média μ, a variância σ² de cada atributo e a priori $P(C_k)$. Assume atributos independentes.

predict: soma log da priori com o log das densidades e escolhe a classe com maior valor.

Somar logaritmos em vez de multiplicar probabilidades evita underflow numérico.

DECISÃO

$$\text{argmax}_k \log P(C_k) + \Sigma_j \log \mathcal{N}(x_j;\ \mu_{kj},\ \sigma^2_{kj})$$

src/classificacao/bayes.py

```python
class NaiveBayes:  # Univariado
    def fit(self, X_train, y_train):
        n_samples, n_features = X_train.shape
        self._classes = np.unique(y_train)
        # ... aloca _mean, _var e _prior com zeros
        for idx, c in enumerate(self._classes):
            X_c = X_train[y_train == c]
            self._mean[idx, :] = X_c.mean(axis=0)
            self._var[idx, :] = X_c.var(axis=0)
            self._prior[idx] = X_c.shape[0] / float(n_samples)

    def predict(self, X_test):
        return np.array([self._predict(x) for x in X_test])

    def _predict(self, x):
        posteriors = []
        for idx, c in enumerate(self._classes):
            prior = np.log(self._prior[idx])  # evita underflow
            posterior = np.sum(np.log(self._pdf(idx, x)))
            posteriors.append(prior + posterior)
        return self._classes[np.argmax(posteriors)]
```

2. Algoritmos — Regressão Linear Múltipla

Solução fechada, com estabilidade numérica.

fit: adiciona uma coluna de 1s (intercepto β_0) e resolve a equação normal uma única vez.

predict: só um produto matricial com os β já calculados. Teste em menos de 1 ms.

A pseudo-inversa de Moore–Penrose (np.linalg.pinv) funciona mesmo quando X^TX não é invertível. Sem hiperparâmetro.

EQUAÇÃO NORMAL

$$\beta = (X^TX)^+ X^Ty \rightarrow \hat{y} = X\beta$$

src/regressao/regressao_linear.py

```python
class MRegression:
    def fit(self, X_train, y_train):
        N = X_train.shape[0]
        # coluna de 1s para o intercepto
        X_train = np.column_stack((np.ones(N), X_train))
        # pseudo-inversa de Moore-Penrose
        self.beta = (np.linalg.pinv(X_train.T @ X_train)
                     @ X_train.T @ y_train)

    def predict(self, X_test):
        N = X_test.shape[0]
        X_test = np.column_stack((np.ones(N), X_test))
        return X_test @ self.beta
```

3. Experimentos — Validação Cruzada K-Fold (k = 5)



Função k_fold no estilo do train_test_split da aula (np.random.seed, np.random.permutation, math.ceil). Em cada fold: padronização Z-score ajustada só no treino → treino → teste. Para cada algoritmo: média ± desvio-padrão das métricas nos 5 folds e tempos de treino e de teste.

```
seed 42 → permutation → 5 partições → 100% dos dados testados uma vez
```

3.2 Métricas de Avaliação

CLASSIFICAÇÃO

$$\text{Acurácia} = (VP + VN) / N$$

$$\text{Precisão} = VP / (VP + FP)$$

$$\text{Recall} = VP / (VP + FN)$$

$$F1 = 2 \cdot P \cdot R / (P + R)$$

Precisão, recall e F1 são médias macro entre as classes.

REGRESSÃO

$$MAE = (1/n) \Sigma |y_i - \hat{y}_i|$$

$$R^2 = 1 - SSres / SStot$$

$$R^2aj = 1 - (1 - R^2)(n - 1)/(n - p - 1)$$

Todas implementadas manualmente com NumPy.

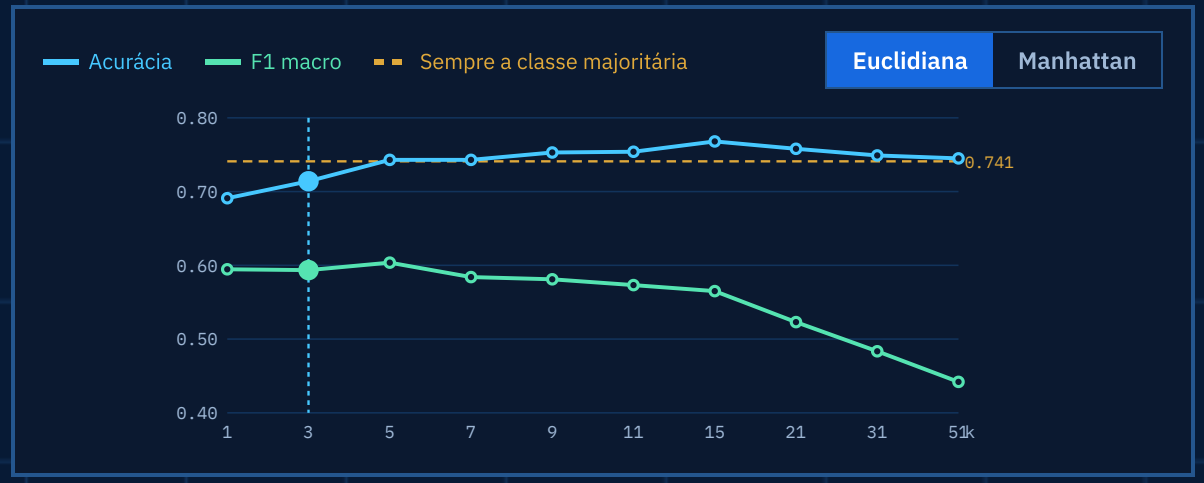
3.3 Resultados de Classificação

AutoUniv-au1-1000 · média ± desvio-padrão em 5 folds · mova o controle para ver o efeito de k

K DO KNN

k = 3

CLASSIFICADOR	ACURÁCIA	PRECISÃO	RECALL	F1-SCORE	TREINO (s)	TESTE (s)*
kNN (Euclidiana) k = 3	0.714 ± 0.027	0.609 ± 0.029	0.588 ± 0.017	0.593 ± 0.020	0.000 ± 0.000	0.475 ± 0.010
kNN (Manhattan) k = 3	0.714 ± 0.027	0.609 ± 0.029	0.588 ± 0.017	0.593 ± 0.020	0.000 ± 0.000	0.422 ± 0.006
Bayes Univariado sem hiperparâmetro	0.722 ± 0.042	0.608 ± 0.068	0.565 ± 0.036	0.566 ± 0.044	0.003 ± 0.006	0.005 ± 0.001



Se aumentar k (kNN Euclidiana, k = 3)

Acurácia: ir para k = 5 melhora `0.714 → 0.743`.
Melhor k para esta métrica: 15 (0.768)

F1 macro: ir para k = 5 melhora `0.593 → 0.604`.
Melhor k para esta métrica: 5 (0.604)

Bayes Univariado: não tem hiperparâmetro (classe NaiveBayes da aula). Resultado fixo: acurácia 0.722, F1 0.566.

* Tempo medido na configuração do trabalho. No kNN, o tempo de teste quase não muda com k: o custo é calcular e ordenar as distâncias para todo o treino.

3.3 Resultados de Regressão

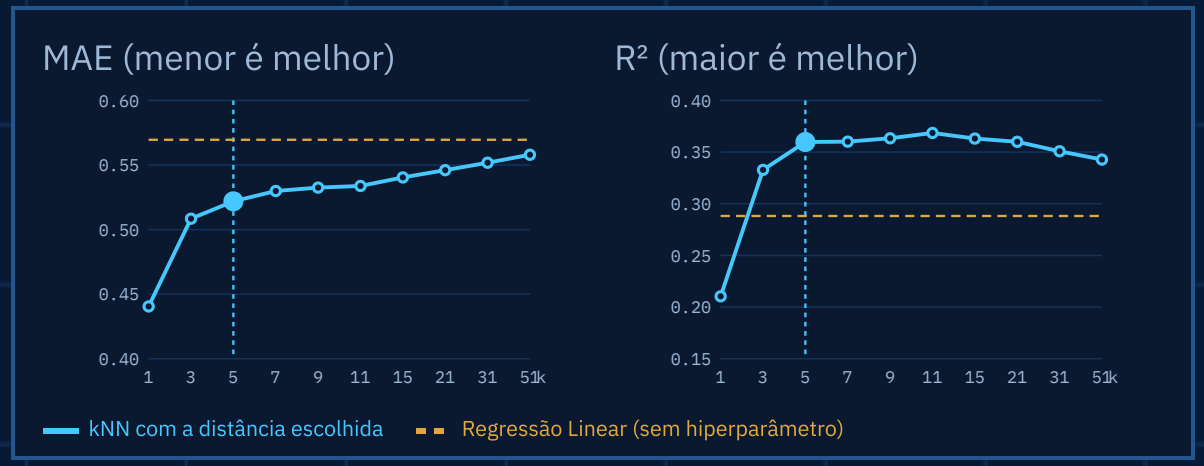
Wine Quality · média ± desvio-padrão em 5 folds · mova o controle para ver o efeito de k

K DO KNN

k = 5

Euclidiana | Manhattan

REGRESSOR	MAE	R²	R² AJUSTADO	TREINO (s)	TESTE (s)*
Regressão Linear sem hiperparâmetro	0.570 ± 0.004	0.288 ± 0.018	0.282 ± 0.018	0.001 ± 0.000	0.000 ± 0.000
kNN (Euclidiana) k = 5	0.525 ± 0.006	0.358 ± 0.016	0.353 ± 0.016	0.000 ± 0.000	19.577 ± 0.120
kNN (Manhattan) k = 5	0.522 ± 0.006	0.360 ± 0.021	0.354 ± 0.021	0.000 ± 0.000	17.586 ± 0.169



Se aumentar k (kNN Manhattan, k = 5)

MAE: ir para k = 7 piora `0.522 → 0.530`.
Melhor k para esta métrica: 1 (0.441)

R²: ir para k = 7 não muda `(0.360)`.
Melhor k para esta métrica: 11 (0.369)

Regressão Linear: não tem hiperparâmetro. A pseudo-inversa dá uma única solução: MAE 0.570, R² 0.288.

* Tempo medido na configuração do trabalho; no kNN ele quase não depende de k.

Análise Crítica: Desempenho vs. Eficiência

CLASSIFICADORES (ACURÁCIA E F1)

Acurácia: Bayes 0.722 ≈ kNN 0.714.

F1 macro: kNN 0.593 > Bayes 0.566. O kNN acerta mais a classe minoritária.

Teste: Bayes 0.005 s vs. kNN ~0.45 s (≈100× mais rápido).

REGRESSORES (R^2 E MAE)

kNN Manhattan: MAE 0.522 e R^2 0.360.

Regressão Linear: MAE 0.570 e R^2 0.288.

Teste: Linear < 1 ms vs. kNN ~18–20 s por fold; treino do kNN ≈ 0 s (só armazena).

kNN: simples e sem suposições sobre os dados, mas a previsão escala mal com o tamanho do treino. Bayes e Regressão Linear: rápidos e interpretáveis, mas assumem independência entre atributos e relação linear.

Ajuste de Hiperparâmetros: aumentar sempre ajuda?

Não. Cada métrica tem seu melhor valor, e passar dele piora o modelo.

K DO KNN · CLASSIFICAÇÃO

A acurácia sobe até k = 15 (0.768), mas o F1 cai de 0.604 (k = 5) para 0.442 (k = 51).
Com k grande o voto é dominado pela classe majoritária (74%): o modelo acerta mais por “chutar” class2 e esquece a class1.
Melhor k pelo F1: 5.

K DO KNN · REGRESSÃO

O R^2 sobe de 0.21 (k = 1) até 0.369 (k = 11, Manhattan) e depois cai: média de vizinhos demais suaviza a previsão.
O MAE é menor em k = 1 (0.441) porque copia notas inteiras: acerta muitas exatamente e erra feio as outras, o que o R^2 penaliza.
Melhor k pelo R^2: 7 a 11.

Aumentar k reduz a variância e aumenta o viés. O ponto ótimo depende da métrica escolhida, por isso a métrica deve ser definida antes de ajustar.

4. Conclusões

Melhor equilíbrio: Bayes Univariado e Regressão Linear.

Melhor desempenho: kNN no F1 da classificação (0.593; 0.604 com k = 5) e kNN Manhattan na regressão (MAE 0.522, R^2 0.360; R^2 0.369 com k = 11).

Mais rápidos: Bayes Univariado (0.005 s de teste) e Regressão Linear (< 1 ms).

Vale a pena? Na classificação, o Bayes entrega acurácia equivalente ~100× mais rápido. Na regressão, o kNN ganha +0.07 de R^2 ao custo de ~18 s por fold; compensa só se a previsão não precisar ser em tempo real.

Limitações: dataset de classificação artificial, ruidoso e desbalanceado (prever sempre a classe majoritária daria 0.741 de acurácia); R^2 moderado no Wine Quality; k escolhido sem validação aninhada.

5. Referências

OPENML. Dataset 1547 (AutoUniv-au1-1000) e Dataset 287 (Wine Quality). Disponível em: https://www.openml.org/

CORTEZ, P. et al. Modeling wine preferences by data mining from physicochemical properties. Decision Support Systems, 47(4), 547–553, 2009.

HARRIS, C. R. et al. Array programming with NumPy. Nature, 585, 357–362, 2020.

BISHOP, C. M. Pattern Recognition and Machine Learning. Springer, 2006.

MAIA, C. M. Material e códigos da disciplina Inteligência Artificial Computacional (kNN, regressão linear múltipla, hold-out). UNIFOR – CCT, 2026.

Python + NumPy